

Number of Connected Components

Problem Context

given:

n nodes labelled 0, 1, 2, ..., n-1

An edge list:

edges = {{u,v}, ...}

Each edge means:

u ----- v

How many disconnected groups (connected components) exist in the graph?

Approach 1 — DFS

Intuition:

- Build the graph using an adjacency list.
- Aadya will try starting DFS from every node.
- If a node has already been visited, it already belongs to some connected component.
- If it has not been visited, then Aadya has discovered an entirely new connected component.
- Increase the answer and traverse the whole component using DFS.

```
class Solution {
public:

    //-----
    // DFS
    // currnode = node Aadya is currently standing on
    //-----

    void dfs(vector<vector<int>>& adj, int currnode, vector<int>& seen){

        seen[currnode] = 1;

        for(auto &nextnode : adj[currnode]){ //saare neighbour ko seen kr lo
            if(!seen[nextnode]){
                dfs(adj, nextnode, seen);
            }
        }
    }

    int countComponents(int n, vector<vector<int>>& edges) {

        vector<vector<int>> adj(n);

        for(auto &it : edges){
            int u = it[0];
            int v = it[1];
            adj[u].push_back(v);
            adj[v].push_back(u);
        }
    }
};
```

```
vector<int> seen(n, 0);
int ans = 0;

// Every unseen node starts a NEW connected component

for(int node = 0; node < n; node++){
    if(!seen[node]){
        ans++;
        dfs(adj, node, seen);
    }
}

return ans;
}

};
```

Approach 2 — BFS

Intuition:

- Exactly the same idea.
- Instead of recursively exploring deeper, Aadya explores one level at a time using a queue.

Code (BFS)

```
class Solution {
public:

    int countComponents(int n, vector<vector<int>>& edges) {

        vector<vector<int>> adj(n);

        for(auto &it : edges){
            int u = it[0];
            int v = it[1];
            adj[u].push_back(v);
            adj[v].push_back(u);
        }

        vector<int> seen(n, 0);
        int ans = 0;

        for(int node = 0; node < n; node++){
            if(seen[node]) continue;

            ans++;
            queue<int> q;
            q.push(node);
            seen[node] = 1;

            while(!q.empty()){
                int currnode = q.front();
                q.pop();

                for(auto &nextnode : adj[currnode]){
                    if(!seen[nextnode]){
                        seen[nextnode] = 1;
                        q.push(nextnode);
                    }
                }
            }
        }
    }
};
```

```
        }  
    }  
    return ans;  
}  
};
```